The “gradient” argument in Pytorch’s “backward” function — explained by examples

Yang Zhang Follow 8 min read · Aug 24, 2019

373 3

This post is some examples for the `gradient` argument in Pytorch's `backward` function. The math of `backward(gradient)` is explained in this tutorialand these threads (thread-1, thread-2), along with some examples. Those were very helpful, but I wish there were more examples on how the numbers in the example correspond to the math, to help me more easily understand. I could not find many such examples so I will make some and write them here, so that I can look back when I forget this in two weeks.

In the examples, I run code in torch, write down the math, and run the

math in numpy, and show that the torch result matches the math/numpy result.

The Jupyter notebook version of this post is on github is better formatted and runnable. You can download it and run the code.

. . .

Here's how Pytorch tutorial explains the math:

Mathematically, if you have a vector valued function $\vec{y} = f(\vec{x})$, then the gradient of $\vec{y}$ with respect to $\vec{x}$ is a Jacobian matrix:

$$J = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{pmatrix}$$

We will make examples of `x` *and* `y=f(x)` *(we omit the arrow-hats of* `x` *and* `y` *above), and manually calculate Jacobian* `J`*.*

Pytorch tutorial goes on with the explanation:

Generally speaking, `torch.autograd` is an engine for computing vector-Jacobian product. That is, given any vector $v = \begin{pmatrix} v_1 & v_2 & \cdots & v_m \end{pmatrix}^T$, compute the product $v^T \cdot J$. If v happens to be the gradient of a scalar function $l = g\left(\vec{y}\right)$, that is, $v = \begin{pmatrix} \frac{\partial l}{\partial y_1} & \cdots & \frac{\partial l}{\partial y_m} \end{pmatrix}^T$, then by the chain rule, the vector-Jacobian product would be the gradient of l with respect to $\vec{x}$:

$$J^T \cdot v = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_n} & \cdots & \frac{\partial y_m}{\partial x_n} \end{pmatrix} \begin{pmatrix} \frac{\partial l}{\partial y_1} \\ \vdots \\ \frac{\partial l}{\partial y_m} \end{pmatrix} = \begin{pmatrix} \frac{\partial l}{\partial x_1} \\ \vdots \\ \frac{\partial l}{\partial x_n} \end{pmatrix}$$

(Note that $v^T \cdot J$ gives a row vector which can be treated as a column vector by taking $J^T \cdot v$.)

The above basically says: if you pass `vᵀ` as the `gradient` argument, then `y.backward(gradient)` will give you not `J` but `vᵀ · J` as the result of `x.grad`.

We will make examples of `vᵀ`, calculate `vᵀ · J` in numpy, and confirm that the result is the same as `x.grad` after calling `y.backward(gradient)` where `gradient` is `vᵀ`.

All good? Let's go.

```
import torch
import numpy as np
from torch import tensor
from numpy import array
```

input is scalar, output is scalar

First, a simple example where `x=1` and `y = x^2` are both scalar.

In pytorch:

```
x = tensor(1., requires_grad=True)
print('x:', x)
y = x**2
print('y:', y)
y.backward() # this is the same as y.backward(tensor(1.))
print('x.grad:', x.grad)
```

Out:

```
x: tensor(1., requires_grad=True)
y: tensor(1., grad_fn=<PowBackward0>)
x.grad: tensor(2.)
```

Now manually calculate Jacobian `J`. In this case `x` and `y` are both scalar (each only has one component `x_1` and `y_1` respectively). And we have

$$J = \left(\frac{\partial y_1}{\partial x_1}\right) = \left(\frac{\partial y}{\partial x}\right) = (2x)$$

In numpy:

```
x = x.detach().numpy()
J = array([[2*x]])
print('J:', J)
```

Out:

```
J: [[2.]]
```

In this example, we did not pass the `gradient` argument to `backward()`, and this defaults to passing the value 1. As a reminder, v^T is our `gradient` with value 1. We can confirm that $v^T \cdot J$ gives the same result as `x.grad`. All good.

```
vᵀ = array([[1,]])
print('vᵀ:', vᵀ)
print('vᵀ · J:', vᵀ@J)
```

Out:

```
vᵀ: [[1]]
vᵀ · J: [[2.]]
```

input is scalar, output is scalar, non-default gradient

We can keep everything else the same but pass a non-default `gradient` with the value 100 to `backward()` instead of the default value 1.

```
x = tensor(1., requires_grad=True)
print('x:', x)
y = x**2
print('y:', y)
gradient_value = 100.
y.backward(tensor(gradient_value))
print('x.grad:', x.grad)
```

Out:

```
x: tensor(1., requires_grad=True)
y: tensor(1., grad_fn=<PowBackward0>)
x.grad: tensor(200.)
```

This is the same as setting the value `100` for v^T, and we can see $v^T \cdot J$ still matches `x.grad`. Still good.

```
x = x.detach().numpy()
J = array([[2*x]])
print('J:', J)

vT = array([[gradient_value,]])
print('vT:', vT)
print('vT · J:', vT@J)
```

Out:

```
J: [[2.]]
v^T: [[100.]]
v^T · J: [[200.]]
```

input is vector, output is scalar

Now let's look at a more interesting example where `x=[x_1,x_2]=[1,2]` is a vector and `y=sum(x)` is a scalar.

```
x = tensor([1., 2.], requires_grad=True)
print('x:', x)
y = sum(x)
print('y:', y)
y.backward()
print('x.grad:', x.grad)
```

Out:

```
x: tensor([1., 2.], requires_grad=True)
y: tensor(3., grad_fn=<AddBackward0>)
x.grad: tensor([1., 1.])
```

Now manually calculate Jacobian `J`. In this since `x` is a vector with components `x_1` and `x_2`, and `y=x_1+x_2` is a scalar. We have

$$J = \left(\frac{\partial y_1}{\partial x_1}, \frac{\partial y_1}{\partial x_2} \right) = \left(1, 1 \right)$$

In numpy:

```
J = array([[1, 1]])
print('J:')
print(J)
```

Out:

```
J:
[[1 1]]
```

In this example, we did not pass the `gradient` argument to `backward()`, and this defaults to passing the value 1, i.e., v^T has value 1. We can confirm that $v^T \cdot J$ gives the same result as `x.grad`.

```
vᵀ = array([[1]])
print('vᵀ:', vᵀ)
print('vᵀ · J:', vᵀ@J)
```

Out:

```
vᵀ: [[1]]
vᵀ · J: [[1 1]]
```

input is vector, output is scalar, non-default gradient

We can keep everything else the same as above but pass a non-default `gradient` with the value 100 to `backward()` instead of the default value 1. Still, `x=[x_1,x_2]=[1,2]` is a vector and `y=sum(x)` is a scalar.

```
x = tensor([1., 2.], requires_grad=True)
print('x:', x)
y = sum(x)
gradient_value = 100.
y.backward(tensor(gradient_value))
print('x.grad:', x.grad)
```

Out:

```
x: tensor([1., 2.], requires_grad=True)
x.grad: tensor([100., 100.])
```

This is the same as setting the value 100 for `vᵀ`, and we can see `vᵀ · J` still matches `x.grad`. Still good.

```
J = array([[1, 1]])
```

```
print('J:')
print(J)

vᵀ = array([[gradient_value,]])
print('vᵀ:', vᵀ)
print('vᵀ · J:', vᵀ@J)
```

Out:

```
J:
[[1 1]]
vᵀ: [[100.]]
vᵀ · J: [[100. 100.]]
```

input is vector, output is vector

Now let's look at an example where both `x=[x_1,x_2]=[1,2]` and `y=3x^2` are vectors.

```
x = tensor([1., 2.], requires_grad=True)
print('x:', x)
y = 3*x**2
print('y:', y)
gradient_value = [1., 1.]
y.backward(tensor(gradient_value))
print('x.grad:', x.grad)
```

Out:

```
x: tensor([1., 2.], requires_grad=True)
y: tensor([ 3., 12.], grad_fn=<MulBackward0>)
x.grad: tensor([ 6., 12.])
```

Now manually calculate Jacobian `J`. In this since `x` is a vector with components `x_1` and `x_2`, and `y=3x^2` is a vector with component `y_1=3x_1^2` and `y_2=3x_2^2`. We have

$$J = \begin{pmatrix} \frac{\partial y_1}{\partial x_1}, \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1}, \frac{\partial y_2}{\partial x_2} \end{pmatrix} = \begin{pmatrix} 6x_1, 0 \\ 0, 6x_2 \end{pmatrix}$$

In numpy:

```
x = x.detach().numpy()
J = array([[6*x[0], 0], [0, 6*x[1]]])
print('J:')
print(J)
```

Out:

```
J:
[[ 6.  0.]
 [ 0. 12.]]
```

In this example, because `y` is a vector, we must pass a `gradient` argument to `backward()`. We pass v^T with the same length as `y` and has values 1. We can confirm that $v^T \cdot J$ gives the same result as `x.grad`.

```
vᵀ = array([gradient_value])
print('vᵀ:', vᵀ)
print('vᵀ · J:', vᵀ@J)
```

Out:

```
vᵀ: [[1. 1.]]
vᵀ · J: [[ 6. 12.]]
```

input is vector, output is vector, non-one gradient

We can keep everything else the same as above but pass a non-default `gradient` with the value [1, 10] to `backward()`.

```
x = tensor([1., 2.], requires_grad=True)
print('x:', x)
y = 3*x**2
print('y:', y)
gradient_value = [1., 10.]
y.backward(tensor(gradient_value))
print('x.grad:', x.grad)
```

Out:

```
x: tensor([1., 2.], requires_grad=True)
y: tensor([ 3., 12.], grad_fn=<MulBackward0>)
x.grad: tensor([  6., 120.])
```

This is the same as setting the value [1,10] for `vᵀ`, and we can see `vᵀ · J` still matches `x.grad`. Still good.

```
x = x.detach().numpy()
J = array([[6*x[0], 0], [0, 6*x[1]]])
print('J:')
print(J)

vᵀ = array([gradient_value])
print('vᵀ:', vᵀ)
print('vᵀ · J:', vᵀ@J)
```

Out:

```
J:
[[ 6.  0.]
 [ 0. 12.]]
vᵀ: [[ 1. 10.]]
vᵀ · J: [[  6. 120.]]
```

input is vector, output is vector — another example

Now let's look at a slightly more complex/full-fledged example where

$$x = [x_1, x_2] = [1, 2]$$

is a vector with 2 components and

$$y = [3x_1^2, x_1^2 + 2x_2^3, 10x_2]$$

is a vector with 3 components.

```
x = tensor([1., 2.], requires_grad=True)
print('x:', x)
y = torch.empty(3)
y[0] = 3*x[0]**2
y[1] = x[0]**2 + 2*x[1]**3
y[2] = 10*x[1]
print('y:', y)
gradient_value = [1., 10., 100.,]
y.backward(tensor(gradient_value))
print('x.grad:', x.grad)
```

Out:

```
x: tensor([1., 2.], requires_grad=True)
y: tensor([ 3., 17., 20.], grad_fn=<CopySlices>)
x.grad: tensor([  26., 1240.])
```

Now manually calculate Jacobian `J`. In this since `x` is a vector with components `x_1` and `x_2`, and

$$y = [y_1 = 3x_1^2, y_2 = x_1^2 + 2x_2^3, y_3 = 10x_2]$$

We have

$$J = \begin{pmatrix} \frac{\partial y_1}{\partial x_1}, \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1}, \frac{\partial y_2}{\partial x_2} \\ \frac{\partial y_3}{\partial x_1}, \frac{\partial y_3}{\partial x_2} \end{pmatrix} = \begin{pmatrix} 6x_1, 0 \\ 2x_1, 6x_2^2 \\ 0, 10 \end{pmatrix}$$

In numpy:

```
x = x.detach().numpy()
J = array([[6*x[0], 0],
           [2*x[0], 6*x[1]**2],
           [0, 10]])
print('J:')
print(J)
```

Out:

```
J:
[[ 6.  0.]
```

```
  [ 2. 24.]
  [ 0. 10.]]
```

In this example, because `y` is a vector, we must pass a `gradient` argument to `backward()`. We pass v^T with the same length as `y` and has values [1., 10., 100.,]. We can confirm that $v^T \cdot J$ gives the same result as `x.grad`.

```
vᵀ = array([gradient_value])
print('vᵀ:', vᵀ)
print('vᵀ · J:', vᵀ@J)
```

Out:

```
vᵀ: [[  1.  10. 100.]]
vᵀ · J: [[  26. 1240.]]
```

extra cases: broadcast/accumulate

But there's more. I've not seen it elsewhere other than here, but when `y` is a scalar, you can actually pass a vector as the `gradient`.

extra cases-1: input is scalar, output is scalar, gradient is vector

As in our very first simple example, let `x=1` and `y = x^2`, both scalar. But instead of a scalar, we can pass a vector of arbitrary length as

`gradient`.

In pytorch:

```
x = tensor(1., requires_grad=True)
print('x:', x)
y = x**2
print('y:', y)
gradient_value = [1., 10., 100., 1000.]
y.backward(tensor(gradient_value))
print('x.grad:', x.grad)
```

Out:

```
x: tensor(1., requires_grad=True)
y: tensor(1., grad_fn=<PowBackward0>)
x.grad: tensor(2222.)
```

With this vector gradient, `backward` accumulates gradient for `x`:

```
x = tensor(1., requires_grad=True)
y = x**2
gradient_value = [1., 10., 100., 1000.]
for v in gradient_value:
    y.backward(tensor(v), retain_graph=True)
    print('x.grad:', x.grad)
```

Out:

```
x.grad: tensor(2.)
x.grad: tensor(22.)
x.grad: tensor(222.)
x.grad: tensor(2222.)
```

In the matrix multiplication universe, this behavior is as if `J` is broadcast to the same length of the `gradient`.

As before, the Jacobian:

$$J = \left(\frac{\partial y_1}{\partial x_1}\right) = \left(\frac{\partial y}{\partial x}\right) = (2x)$$

In numpy:

```
x = x.detach().numpy()
J = array([[2*x]])
print('J:', J)

J_broadcast = np.repeat(J, len(gradient_value), axis=0)
print('J_broadcast:')
print(J_broadcast)
```

Out:

```
J: [[2.]]
```

```
J_broadcast:
[[2.]
 [2.]
 [2.]
 [2.]]
```

We can confirm that `vᵀ · J_broadcast` gives the same result as `x.grad`. All good.

```
vᵀ = array([gradient_value])
print('vᵀ:', vᵀ)
print('vᵀ · J_broadcast:', vᵀ@J_broadcast)
```

Out:

```
vᵀ: [[   1.   10.  100. 1000.]]
vᵀ · J_broadcast: [[2222.]]
```

extra cases-2: input is vector, output is scalar, gradient is vector

Here's another example of broadcast/accumulate. As in our second example, let `x=[x_1,x_2]=[1,2]` is a vector and `y=sum(x)`. But instead of a scalar, we can pass a vector of arbitrary length as `gradient`.

```
x = tensor([1., 2.], requires_grad=True)
print('x:', x)
y = sum(x)
```

```
print('y:', y)
gradient_value = [1., 10., 100., 1000.]
y.backward(tensor(gradient_value))
print('x.grad:', x.grad)
```

Out:

```
x: tensor([1., 2.], requires_grad=True)
y: tensor(3., grad_fn=<AddBackward0>)
x.grad: tensor([1111., 1111.])
```

With this vector gradient, `backward` accumulates gradient for `x`:

```
x = tensor([1., 2.], requires_grad=True)
y = sum(x)
gradient_value = [1., 10., 100., 1000.]
for v in gradient_value:
    y.backward(tensor(v), retain_graph=True)
    print('x.grad:', x.grad)
```

Out:

```
x.grad: tensor([1., 1.])
x.grad: tensor([11., 11.])
x.grad: tensor([111., 111.])
x.grad: tensor([1111., 1111.])
```

In the matrix multiplication universe, this behavior is as if `J` is broadcast to the same length of the `gradient`.

$$J = \left(\frac{\partial y_1}{\partial x_1}, \frac{\partial y_1}{\partial x_2} \right) = \left(1, 1 \right)$$

In numpy:

```
x = x.detach().numpy()
J = array([[1, 1]])
print('J:', J)

J_broadcast = np.repeat(J, len(gradient_value), axis=0)
print('J_broadcast:')
print(J_broadcast)
```

Out:

```
J: [[1 1]]
J_broadcast:
[[1 1]
 [1 1]
 [1 1]
 [1 1]]
```

We can confirm that `vᵀ · J_broadcast` gives the same result as `x.grad`.

```
vᵀ = array([gradient_value])
print('vᵀ:', vᵀ)
print('vᵀ · J_broadcast:', vᵀ@J_broadcast)
```

Out:

```
vᵀ: [[   1.   10.  100. 1000.]]
vᵀ · J_broadcast: [[1111. 1111.]]
```

That's it.

Hope this helps. Since there's some manual calculation of gradients here, if I made mistakes, please let me know so I can correct.

Machine Learning | Pytorch | Deep Learning | Data Science